

Class starts at 9:10pm

Type Safety & Validation with Hydrite

Class starts at 9:10 PM

Agenda

- why type safety matters ✓
- Serialization for agent I/O
- Custom validators for URI output
- Decorators & Content Managers

Input $\rightarrow$ [LLM] $\rightarrow$ output
prompt / user query

output
Type of task
output can
be a web

Software
system
 $\downarrow$
deterministic

i) fixed structure

ii) proper data validation

this is where pydantic comes into picture

id = 100 $\checkmark$
 $\downarrow$
int

&

id = "100"
 $\downarrow$
int

?

$\rightarrow$ pydantic would
make it work

Why Type safety matters?

Any hallucination that happens can generate invalid data which can corrupt or damage the system

Runtime Errors vs Silent Data Corruption

- **Runtime Errors:** Loud failures that crash early and are easier to detect.
- **Silent Data Corruption:** Wrong data flows through the system and produces wrong results without any error.

Example:
A price stored as 100 instead of 10.0 won't crash your app, but can lead to wrong orders, reports, or financial loss.

- Validate all inputs and outputs at the boundaries of your system.

- Do not trust external data (APIs, users, files, or models).
- Enforce schemas early so invalid data never reaches your core logic.



Principle:
Validate Early.
Fail Fast. Stay Safe.



Type safety and validation turn unpredictable data into reliable, trustworthy inputs — leading to more robust, secure, and maintainable AI systems.



2

Pydantic V2 Foundations

Define, validate and serialize data with Python type hints.



Creating BaseModel

Models in Pydantic are created by inheriting from BaseModel.

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str
```

Field Types

Use standard Python types.

```
class Product(BaseModel):
    id: int
    name: str
    price: float
    in_stock: bool
    tags: List[str]
```

Optional Fields

Use Optional[T] or None for optional values.

```
from typing import Optional

class User(BaseModel):
    name: str
    age: Optional[int] = None
    bio: str | None = None
```

Default Values

Provide default values for fields when needed.

```
class Settings(BaseModel):
    env: str = "Dev"
    retries: int = 3
    timeout: float = 10.0
```

Strict vs Lax Type Coercion

Lax mode (default) tries to coerce types. Strict mode does not.

```
from pydantic import BaseModel
from typing import List

class User(BaseModel):
    id: int
    name: str
    email: str

# Lax mode (default)
user = User(id="1", name="test") # id is int()

# Strict mode
from pydantic import StrictInt
class User(BaseModel):
    model_config = {"strict": True}
    id: StrictInt
    name: str

# Validation error
X User(id="1", name="test") # ValidationError
```

Annotated

Use Annotated for richer metadata and constraints.

```
from typing import Annotated
from pydantic import Field

Age = Annotated[int, Field(gt=18, lt=100)]

class Person(BaseModel):
    name: str
    age: Age
```

Field()

Add metadata, descriptions, examples and more.

```
class Item(BaseModel):
    name: str = Field(
        description="Name of the item",
        min_length=1,
        examples=["Book"]
    )
    price: float = Field(gt=0, description="Price in USD")
```

Numeric Constraints

Enforce limits on numbers.

```
class Score(BaseModel):
    percentage: Annotated[float, Field(gt=0, lt=100)]
    count: Annotated[int, Field(gt=0, lt=1000)]
    rating: Annotated[float, Field(gt=0, lt=5)]
```

Validation Errors

Pydantic provides clear and structured error messages.

```
from pydantic import ValidationError

try:
    score = Score(percentage=150)
except ValidationError as e:
    print(e)
```

```
1 validation error for Score
percentage
  Input should be less than or equal to 100
  (Input: 150, type=value_error, input_type=int)
For further information visit https://errors.pydantic.dev/...
```



Key Takeaway

Pydantic V2 lets you define data models using Python type hints, enforce rules, and get reliable, validated data with minimal code.



Type safe.
Runtime safe.
Production safe.



Serialisation for agent I.O

What is Serialization?

Why It Matters

- ## Core Methods

model_dump()

```
model_dump_json()
```

`model_validate()`
Create and validate a model from a dictionary or JSON-like data.

1 Create Model

2 Serialize (Dump)

```
from pydantic import
```

```
user_dict = user.model_dump()
print(user_dict)

# Output(dict)
{
  'id': 1,
  'name': 'Ani',
  'email': 'ani@example.com',
  'is_active': True
}

user_json = user.model_dump_json()
print(user_json)

# Output (JSON)
{'id':1,'name':'Ani',
'email':'ani@example.com',
'is_active':true}
```

Common Use Cases

 **Database Storage**
Store as JSON documents in MongoDB, PostgreSQL (JSONB)

 **Message Queues**
Publish validated JSON to Kafka, RabbitMQ, SQS, etc.



AI Agents & Tools
Pass structured data between agents and tools safely.

3 Deserialize (Validate)

```
data = {
    "id": 2,
    "name": "Riya",
    "email": "riya@example.com",
    "is_active": False
}

user2 = User.model_validate(data)
print(user2)

# Output (Model Instance)
id=2 name='Riya'
email='riya@example.com'
is_active=False
```



Key Takeaway

Pydantic V2 makes serialization simple, consistent, and safe – ensuring data integrity across your systems.



- Validate on input.
- Serialize on output.
- Trust the schema.



